

VLIW Bundling and Hazard Handling

How `bundler.py` Packs an In-Order VLIW Pipeline Safely

APARA C Compiler Project

June 22, 2026

1 Introduction: Why Bundling Needs a Hazard Algorithm at All

APARA is a VLIW (Very Long Instruction Word) machine: up to 8 instructions are packed into one *bundle* and issued together, in a single cycle. `codegen.py` initially emits one instruction per bundle (the simplest possible correct output); `bundler.py` then repacks that flat stream into denser, multi-instruction bundles wherever it is safe to do so. “Safe” is the entire problem this report covers.

The pipeline is strictly in-order and bundle-at-a-time: bundle N fully executes — every instruction inside it reads its operands, computes, and writes its result — before bundle $N + 1$ begins. There is no out-of-order execution and no result forwarding *within* a bundle from one instruction to another. Crucially, *within* a single bundle, the hardware reads **all** operands for **all** instructions in that bundle first, and only afterward commits **all** writes. This one fact — reads before writes, for the whole bundle, not instruction-by-instruction — is the entire reason hazard analysis is needed, and it is what Section 3 explains in detail.

2 The Hazard Taxonomy

`bundler.py` tracks six distinct hazard conditions. The first three are the classical register hazard triad; the rest are APARA-specific consequences of how memory and control flow interact with bundle-at-a-time, read-before-write execution.

Two further, non-hazard rules close any open bundle unconditionally: a **label** forces a new bundle to start (the assembler can only attach one label per bundle), and a **control-transfer instruction** (`$goto`, `$call`, `$return`, `$halt`) must be the *last* instruction in whatever bundle it ends up in — once one has been added, the next instruction always starts a fresh bundle. Finally, a bundle is capped at **8 instructions**, the hardware’s bundle-width limit.

3 Why RAW Specifically: a Consequence of the In-Order, Read-Then-Write Pipeline

This is worth deriving explicitly, because it is the rule that does almost all of the work in practice (Section 5 shows a real loop where every single split is a RAW split). Consider two instructions A and B that the compiler is considering for the *same* bundle, where A computes a value into register `$rX` and B needs to read `$rX` expecting A’s freshly-computed result:

- **If A and B share a bundle:** the hardware reads every operand for every instruction in that bundle *before* committing any write. B’s read of `$rX` therefore observes whatever value `$rX` held *before* the bundle started — A’s write has not happened yet from B’s point of view, even though both are issued “together.” B silently gets a stale operand.

Table 1: The six conditions `_pack_bundles` checks before adding an instruction to the currently-open bundle

Hazard	Condition	Verdict
RAW (Read-after-Write)	new instr. reads a register the open bundle already writes	split (Section 3)
WAW (Write-after-Write)	new instr. writes a register the open bundle already writes	split — the bundle’s two writers would race for the same destination; whichever the hardware resolves last wins, non-deterministically from the compiler’s point of view
WAR (Write-after-Read)	new instr. writes a register the open bundle only <i>read</i>	safe — all reads in the bundle complete before any write, so the earlier read never observes the later write
Memory RAW/WAW	new instr. accesses (\$ld or \$st) the exact [base+offset] the open bundle already \$st’d to	split — same read-before-write reasoning as registers, applied to a DMEM address instead of a register name
Phase hazard	new instr. writes a register an already-bundled \$ld/\$st reads, and the new instruction is <i>not itself</i> a memory op	split — the physical aligner schedules memory ops into <i>later</i> execution slots than ALU/\$set within one bundle regardless of program-text order, so an ALU write “meant” to land after the load would actually land before it
Call+SP hazard	new instr. is \$call and the open bundle already writes the stack pointer	split — the exact timing interaction is unconfirmed by hardware test, so the two are conservatively never co-bundled

- **If A and B are in consecutive bundles instead:** bundle N (containing A) fully completes — including committing A’s write to `$rX` — before bundle $N + 1$ (containing B) even begins reading its operands. B now correctly observes A’s result.

This is exactly what “in-order, bundle-at-a-time” buys and costs: there is no mechanism (no forwarding network, no out-of-order completion) for a same-cycle producer to reach a same-cycle consumer, so the *only* correct way to sequence a true data dependency is to put the dependent instruction in a strictly later bundle. WAR has the opposite shape and is therefore safe by the same reasoning: if A merely *reads* `$rX` and B *writes* it, A’s read (at the start of the shared cycle) is already resolved before B’s write (at the end of the same cycle) ever happens, so no staleness is possible. WAW is unsafe for an unrelated reason — not staleness, but a genuine race for the same destination register with no defined winner — so it is also split, conservatively.

4 The Bundling Algorithm

`bundle_mcode` (`bundler.py:398`) runs a strict four-stage pipeline: parse the flat, one-instruction-per-bundle mcode text into a list of instruction records (`_parse_flat`, with each record’s reads/writes/memory-access computed by `_parse_deps`); greedily pack that list into bundles (`_pack_bundles`, the algorithm below); collapse any bundle that ended up with more than one label (`_merge_duplicate_labels`); and serialize the result back to mcode text (`_emit_bundles`). The packing stage is the one that implements every hazard rule from Section 2:

Listing 1: `_pack_bundles`, `bundler.py:279–349` (essence)

```

1 function PACK-BUNDLES(flat_instructions):
2   open_bundle <- empty
3   c_writes, c_mem_writes, c_mem_reads <- empty sets
4   c_ctrl <- False
5
6   for instr in flat_instructions:
7     split <- False
8     if open_bundle is not empty:
9       if instr has a label:                                split <- True
10      else if c_ctrl == True:                                split <- True
11      else if instr.reads intersects c_writes:               split <- True
12      // RAW
13      else if instr.writes intersects c_writes:               split <- True
14      // WAW
15      else if instr.mem_access in c_mem_writes:               split <- True
16      // mem RAW/WAW
17      else if not instr.is_mem and (instr.writes intersects c_mem_reads):
18                                                                    split <- True
19      // phase hazard
20      else if instr.is_call and SP_REG in c_writes:           split <- True
21      // call+SP
22      else if len(open_bundle) >= 8:                           split <- True
23      // bundle full
24
25     if split:
26       FLUSH(open_bundle); open_bundle <- empty (reset all tracking sets)
27
28     APPEND instr to open_bundle
29     c_writes <- c_writes union instr.writes
30     if instr.mem_write exists: c_mem_writes <- c_mem_writes union {instr.
31 mem_write}
32     if instr.is_mem: c_mem_reads <- c_mem_reads union instr.reads
33     c_ctrl <- c_ctrl or instr.is_ctrl
34
35   FLUSH(open_bundle) // final partial bundle
36   return all flushed bundles

```

The checks run in a **fixed order** and the **first** one that matches wins — this matters because, for example, a label always forces a split regardless of whether a RAW hazard is also present, and the bundle-full check only ever fires as a last resort once every hazard check has already passed. Tracking is purely additive within one open bundle: `c_writes` accumulates every register written so far in the bundle (used for both RAW and WAW checks against the *next candidate* instruction), and is discarded entirely on every split.

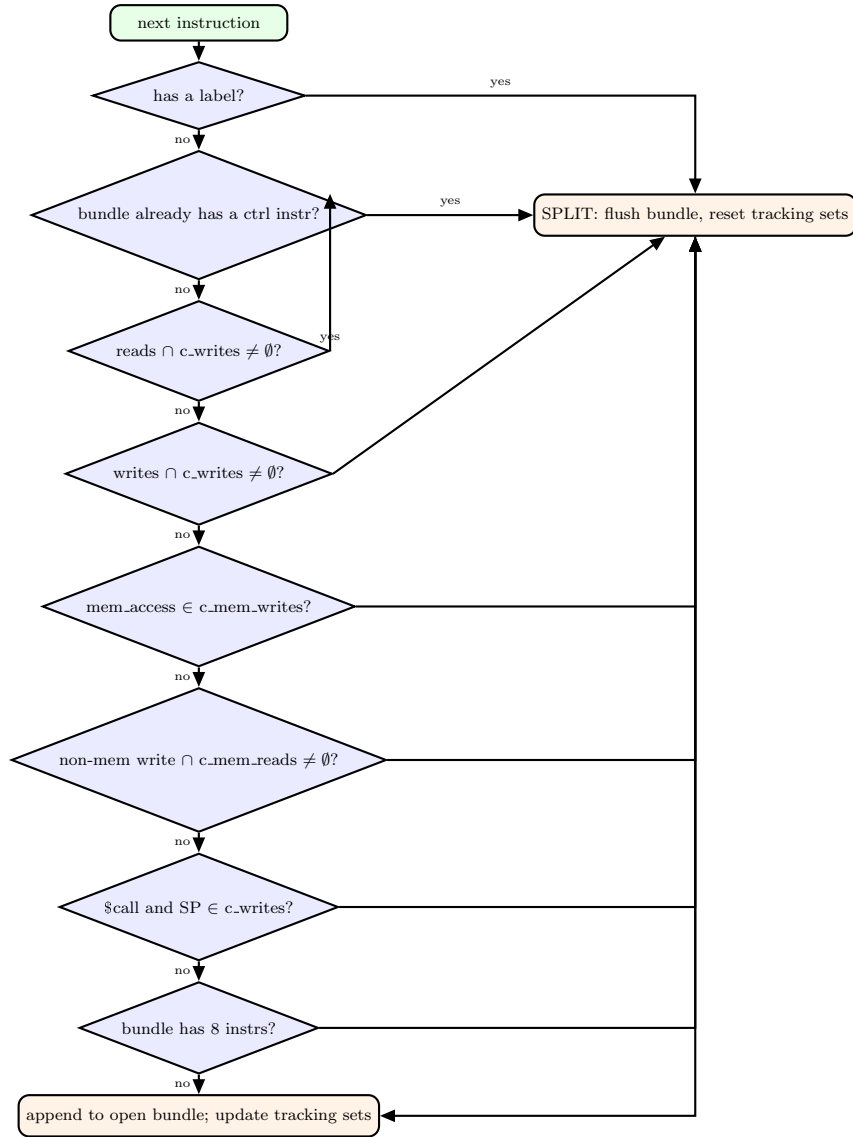


Figure 1: The per-instruction decision chain inside `_pack_bundles`. Every “yes” exit leads to the same SPLIT action; the chain only reaches `add` once every check has answered “no.” This is a direct, one-to-one picture of the `if/elif` chain in Listing 1.

5 A Fully Hardware-Verified Worked Example

The loop `i = 0; while (i < 16) { i = i + 1; }` compiles (Section 4 of the companion branch/loop report traces the IR) to the following 9 bundles. Every split below is annotated with the exact rule from Section 2 that forced it.

Listing 2: Bundled mcode, with the hazard forcing each split

```

1 wc_1:
2 ||
3     + $r2 ($i64) $r26 -8          // B1: entry, no predecessor
4 ;
5 ||
6     $ld ($i32) $r3 [$r2 + 0]      // B2: RAW on $r2 (written in B1) forces the split;
7     + $r4 ($i64) $r0 16          // the two instrs here are mutually independent,
8 ;                                // so they pack into one bundle together
9 ||
    
```

```

10  - $r5 ($i64) $r4 $r3          // B3: RAW on $r4 AND $r3 (both written in B2)
11  ;
12  ||
13  ? ($i64) $r5 > $goto wb_2    // B4: RAW on $r5 (written in B3)
14  ;
15  ||
16  ? ($i64) $r0 == $goto we_3    // B5: forced by "ctrl ends bundle" -- B4 already
17  ;                             // contains a control-transfer instruction
18  wb_2:
19  ||
20  + $r4 ($i64) $r26 -8          // B6: forced by the wb_2 LABEL (not a register
    hazard)
21  ;
22  ||
23  $ld ($i32) $r5 [$r4 + 0]      // B7: RAW on $r4 (written in B6)
24  ;
25  ||
26  + $r6 ($i64) $r5 1           // B8: RAW on $r5 (written in B7); its second instr
27  + $r7 ($i64) $r26 -8          // is independent, so it packs in alongside
28  ;
29  ||
30  $st ($i32) [$r7 + 0] $r6      // B9: RAW on $r7 (written in B8). The trailing
31  ? ($i64) $r0 == $goto wc_1    // unconditional jump has NO register dependency
32  ;                             // on the store, so it shares this bundle --
33  we_3:                         // it only has to be LAST, not alone.

```

This single 9-bundle loop exercises every rule in this report: RAW forces 6 of the 8 splits, the label rule forces one (B6), and the “ctrl must be last, not necessarily alone” rule is demonstrated twice — once where a control-transfer instruction is genuinely alone (B4/B5, since nothing independent was available), and once where it shares a bundle with an unrelated, independent store (B9).

6 Summary

Bundling in this compiler reduces to one question, asked once per instruction against whatever is already in the currently-open bundle: *does adding this instruction change what any already-bundled instruction would observe, given that the whole bundle reads its operands before any of it writes?* RAW and WAW answer “yes” and force a split; WAR answers “no” and is left alone; the memory, phase, and call+SP rules are the same question applied to DMEM addresses, to the physical aligner’s instruction-class reordering, and to one specific hardware-unconfirmed interaction, respectively. Labels and control-transfer placement are not hazards at all, but hard structural requirements of the assembler and the ISA. The result, as Section 5 shows on real, hardware-verified output, is that bundle density is rarely limited by the algorithm’s own conservatism — it is limited by how much genuine, back-to-back data dependency the compiled code actually contains.